

1. Resumen Ejecutivo

Auditoría técnica del proyecto Angular **SIAF-RP** basada en inspección directa de los 71 archivos de componentes, servicios y configuración. Todos los datos son verificados — ningún punto se asume sin evidencia de código.

71	Componentes totales	71/71	Standalone (100%)	8/8	Rutas lazy-loaded	~12 k	Líneas de código
100%	OnPush detection	11	Con constructor DI	1	Servicios activos	2	Barrels vacíos

2. Estructura de Directorios

src/app/core/	Auth (PermissionService, roleGuard), modelos y configuración global
src/app/features/	8 páginas lazy-loaded: login, otp, virtual-desk, adj-seat-*, chart-accounts-*
src/app/layout/	Navbar, Sidebar, SidePanel — compartidos entre features
src/app/shared/ui/	59 componentes base del design system
src/app/shared/components/	2 componentes compuestos transversales
src/app/shared/types/	Tipos e interfaces compartidos
src/app/shared/utils/	Funciones utilitarias
src/styles/	Sistema de tokens CSS en 3 capas (Figma → semántico → Tailwind)

3. Arquitectura Angular

Standalone Components

71 de 71 componentes son standalone (standalone: true en el @Component decorator). La app arranca con bootstrapApplication() — sin AppModule ni NgModules.

Lazy Loading

Ruta	Componente
/login	LoginComponent
/otp-verification	OtpVerificationComponent
/panel	VirtualDeskComponent
/asiento-ajuste/documentos	AdjustmentSeatDocumentsComponent
/asiento-ajuste/solicitud	AdjustmentSeatRequestComponent
/asiento-ajuste/formulario	AdjustmentSeatFormComponent
/plan-cuentas/documentos	ChartAccountsDocumentsComponent
/plan-cuentas/solicitud	ChartAccountsRequestComponent

Change Detection

ChangeDetectionStrategy.OnPush en el 100% de los componentes (71/71). Ningún componente usa la estrategia Default.

4. Inyección de Dependencias

El proyecto mezcla dos patrones de DI. La mayoría usa el moderno **inject()**, pero **11 componentes aún usan constructor-based DI** — patrón que debería migrarse para mantener consistencia.

Componentes con constructor DI (a migrar)

Archivo	Inyecciones en constructor
features/login/login.component.ts	Router
features/virtual-desk/virtual-desk.component.ts	Router
features/adjustment-seat-form/...component.ts	Router
features/adjustment-seat-request/...component.ts	Router, ChangeDetectorRef
features/chart-accounts-request/...component.ts	Router
shared/ui/documents-records-page/...component.ts	Router
shared/ui/text-field/text-field.component.ts	ChangeDetectorRef
shared/ui/radio/radio.component.ts	ChangeDetectorRef
shared/ui/input/input.component.ts	ChangeDetectorRef
shared/ui/checkbox/checkbox.component.ts	ChangeDetectorRef

shared/ui/create-document/create-document.component.ts

ChangeDetectorRef

La migración es mecánica: eliminar el constructor y declarar `private readonly router = inject(Router)` como propiedad de clase.

Patrón correcto (inject) — ya usado en otros componentes

roleGuard y otros usan el patrón moderno: `private readonly permissionService = inject(PermissionService)`. Los 11 componentes listados arriba deben seguir este mismo patrón.

5. Estado Reactivo

Signals

Solo 4 archivos usan `signal()` activamente. El uso de `computed()` se limita a 1 archivo y `effect()` no se usa en ningún componente.

API	Archivos	Ejemplos
<code>signal()</code>	4	<code>periodoPanelOpen</code> , <code>selectedPeriodo</code> , <code>periodoSearch</code> , <code>loginTab</code>
<code>computed()</code>	1	<code>filteredExternalEntities</code> en <code>chart-accounts-request</code>
<code>effect()</code>	0	—

RxJS

Cero usos de `Observable`, `Subject`, `BehaviorSubject` o `subscribe()` en todo el proyecto. El proyecto es 100% signals para estado reactivo.

6. Lifecycle Hooks

Hook	Archivos	Comentario
<code>OnChanges / ngOnChanges</code>	6	<code>process-menu-tree</code> , <code>text-field</code> , <code>date-time-picker</code> , <code>modal</code> , <code>documents-records-page</code> , <code>custom-filter</code>
<code>OnInit / ngOnInit</code>	2	<code>process-menu-tree</code> , <code>date-time-picker</code>
<code>OnDestroy / ngOnDestroy</code>	1	<code>uploader.component.ts</code>
<code>ControlValueAccessor</code>	4	<code>radio</code> , <code>input</code> , <code>checkbox</code> , <code>text-field</code> (integración con formularios Angular)

`OnChanges` en componentes de UI es esperado (reaccionar a cambios de `@Input`). `ControlValueAccessor` es el patrón correcto para integración con `ReactiveForms`.

7. Uso de forwardRef

`forwardRef` aparece en 5 archivos con dos propósitos distintos. Solo uno es un anti-patrón; los otros 4 son uso correcto.

Archivo	Uso	Evaluación
adjustment-seat-form.component.ts	forwardRef() => ReadonlyCardComponent), forwardRef() => ReadonlyLineComponent)	■ Anti-patrón: componentes definidos inline en el mismo archivo
radio.component.ts	useExisting: forwardRef() => RadioComponent)	✓ Correcto: NG_VALUE_ACCESSOR
input.component.ts	useExisting: forwardRef() => InputComponent)	✓ Correcto: NG_VALUE_ACCESSOR
checkbox.component.ts	useExisting: forwardRef() => CheckboxComponent)	✓ Correcto: NG_VALUE_ACCESSOR
text-field.component.ts	useExisting: forwardRef() => TextFieldComponent)	✓ Correcto: NG_VALUE_ACCESSOR

8. Archivos más Grandes

Archivos con más de 300 líneas son candidatos a refactorización. Más de 500 líneas es un problema claro de responsabilidades mixtas.

#	Archivo	Líneas	Estado
1	adjustment-seat-request.component.ts	1,388	■ Refactorizar
2	documents-records-page.component.ts	944	■ Refactorizar
3	chart-accounts-request.component.ts	784	■ Refactorizar
4	tray-documents-view.component.ts	583	■ Revisar
5	adjustment-seat-form.component.ts	574	■ Revisar
6	create-document.component.ts	490	■ Revisar
7	modal.component.ts	416	■ Revisar
8	process-menu-tree.component.ts	390	■ Revisar
9	date-time-picker.component.ts	360	■ Aceptable
10	text-field.component.ts	342	■ Aceptable

9. Sistema de Design Tokens

Los tokens siguen una arquitectura de 3 capas desde Figma hasta Tailwind. figma-tokens.css es generado automáticamente — no editar manualmente.

Capa	Archivo	Responsabilidad
------	---------	-----------------

1 — Paleta	figma-tokens.css	Tokens primitivos generados por Figma. Colores, tipografía, radios, sombras.
2 — Semántico	tokens/generated/tokens.css	Variables con significado: --sys-color-bg-brand-primary, --sys-color-text-*, etc.
3 — Tailwind	tokens/generated/tailwind.tokens.css	Bloque @theme que expone los tokens como utilities: bg-brand-primary, text-*, etc.
Extra	src/styles.css	Alias personalizados (:root) y override de @theme. Ej: --ref-color-solid-primary-50.
Temas	tokens/themes/light.css + dark.css	Variables de tema claro y oscuro (dark mode preparado).

10. Servicios y Guards

El proyecto tiene un único servicio activo. No existe capa de API ni HttpClient. Todos los datos son hardcodeados en componentes y archivos de configuración.

Archivo	Scope	Responsabilidad
core/auth/permission.service.ts	root (singleton)	Gestión de rol actual con signal. Métodos: setRole(), hasRole(), can(), canAny(). Rol inicial hardcodeado: "creator".
core/auth/role.guard.ts	Funcional (CanActivateFn)	Protege rutas según el rol. Usa inject(PermissionService) e inject(Router). Patrón moderno correcto.

Falta: AuthService (login/logout/token), HttpClient, servicios de datos (DocumentService, etc.)

11. Barrel Files (index.ts)

Archivo	Exports	Estado
core/index.ts	3	✓ Activo
core/auth/index.ts	3	✓ Activo
core/config/index.ts	1	✓ Activo
core/models/index.ts	1	✓ Activo
layout/index.ts	3	✓ Activo
shared/index.ts	5+	✓ Re-exporta sub-barrels
shared/ui/index.ts	44	✓ Activo
shared/components/index.ts	2	✓ Activo
shared/types/index.ts	2	✓ Activo

shared/utils/index.ts	2	✓ Activo
shared/directives/index.ts	0	■ Vacío — solo placeholder
shared/pipes/index.ts	0	■ Vacío — solo placeholder

12. Problemas y Plan de Acción

■ ALTA Migrar 11 componentes de constructor DI a inject()

Login, virtual-desk, adjustment-seat-form, adjustment-seat-request, chart-accounts-request, documents-records-page usan constructor(private router: Router). text-field, radio, input, checkbox, create-document usan constructor(private cdr: ChangeDetectorRef). Migración mecánica: eliminar constructor y declarar como propiedad inject().

■ ALTA Descomponer componentes con más de 500 líneas

adjustment-seat-request.component.ts (1,388 líneas), documents-records-page (944) y chart-accounts-request (784) mezclan template, estado local y lógica de negocio. Solución: extraer subcomponentes y mover estado a servicios o signals.

■ ALTA Sacar componentes inline de adjustment-seat-form

ReadonlyCardComponent y ReadonlyLineComponent están definidos dentro del mismo archivo que AdjustmentSeatFormComponent usando forwardRef para referenciarse mutuamente. Cada componente debe vivir en su propio archivo .component.ts.

■ MEDIA Crear capa de servicios HTTP

Solo existe PermissionService. El rol inicial está hardcodeado como "creator". No hay AuthService, HttpClient ni servicios de datos. Al conectar con un backend real será necesario agregar esta capa.

■ MEDIA Activar el path alias @siaf/ui

tsconfig.json define "@siaf/ui/*": ["src/app/shared/ui/*"] pero ningún archivo lo usa. Adoptarlo eliminaría rutas relativas largas como ../../../../shared/ui/button/...

■ MEDIA Centralizar layout (Navbar + Sidebar) en un shell route

Ambos componentes se declaran en cada feature de forma independiente. Un layout shell en app.routes.ts con children evitaría la duplicación.

■ BAJA Limpiar barrels vacíos (directives y pipes)

shared/directives/index.ts y shared/pipes/index.ts solo tienen un comentario. Si no hay implementación planificada a corto plazo, eliminar las carpetas.

■ BAJA Ampliar el uso de Signals

Solo 4 de 71 componentes usan signal(). Para el estado local de componentes (loading, toggles, filtros) es preferible signals sobre variables de clase planas.

13. Scorecard de Arquitectura

Dimensión	Estado	Nota
Standalone Components	✓ 71/71	Óptimo
Lazy Loading	✓ 8/8 rutas	Óptimo
Change Detection (OnPush)	✓ 100%	Óptimo
RxJS	✓ Sin uso	Óptimo — signals puro
Signals (signal/computed)	■ Solo 4 archivos	Poco adoptado
Inyección de dependencias	■ Mixto (11 con constructor)	Inconsistente
Tamaño de componentes	✗ 3 archivos >700 líneas	Problema
Componentes inline	✗ 1 archivo con 3 componentes	Anti-patrón
Servicios / API layer	✗ Solo 1 servicio (hardcoded)	Faltante
Path aliases en uso	✗ Definido pero no usado	Pendiente
Barrel files	■ 2 vacíos	Menor
Sistema de tokens CSS	✓ 3 capas Figma→Tailwind	Óptimo